Restructured

AI-Based ECG Diagnostic Assistant - audit, refactor and verification

Prepared: 2026-10-28

Repository: AI-Based-ECG-Diagnostic-Assistant

Scope: replace the doctor-centric hospital dashboard with a focused ECG workflow without destroying existing patient or ECG data.

1. Executive summary

The repository was refactored from a hospital-management demo into an ECG screening tool. The working parts were kept: Django authentication, device tokens, the FastAPI ingestion pipeline, the WebSocket live view, the simulator contract, the uPlot chart and the existing visual system. The doctor/patient-account workflow was removed, and three missing capabilities were implemented: a standalone patient registry, deterministic DSP with an optional inference interface, and versioned PDF reports.

The final workflow is: operator logs in; finds or registers a patient; reviews previous sessions and reports; starts a recording on a device; watches the live waveform; stops the recording; the backend computes measurements and optional AI; the operator generates and downloads a PDF; everything remains attached to the patient's permanent ID.

Verification is not a compile-only claim. A simulator run uploaded 12 ECG chunks (2,400 samples) and 36 scalar readings with zero failures. The stored signal produced a measured heart rate of 72.9 bpm from 15 detected R-peaks, a quality score of 100/100, and a two-page PDF report. No AI model is bundled, so every analysis and report explicitly says AI is unavailable and contains no fabricated class or probability.

2. Phase 1 audit

2.1 What existed before the refactor

The project had two backends and one React application. Django owned User (roles patient/doctor/admin), PatientProfile (a one-to-one extension of a patient login), DoctorProfile, DoctorAssignment, Device, Device.assigned_patient and Session. FastAPI owned the telemetry schema: a known_sessions cache with a doctor_ids array, device heartbeats, scalar readings, ECG chunks and a revocation list. The React app had login, dashboard, patients, devices, sessions, live monitor, recording viewer and an admin patient/access screen. The simulator could replay synthetic or MIT-BIH ECG into the ingestion endpoints. There was no DSP, no inference, no analysis persistence and no PDF report.

2.2 Findings that shaped the plan

- A patient was a login account. There was no permanent registry record independent of authentication, and phone number was not modelled at all.
- Doctor assignment was enforced in three places: Django querysets, the session serializer and FastAPI's doctor_ids viewer rule. Removing the UI alone would have left three inconsistent permission models.
- Session.patient pointed at PatientProfile, Device.assigned_patient pointed at PatientProfile, and both IDs could be preserved by copying each profile into the new registry with the same primary key.
- The FastAPI session cache stored doctor_ids; it had to be dropped so the schema matched the new access model.
- The FastAPI Alembic migration used sa.func.now() for timestamp defaults, which rendered DEFAULT (now()) on SQLite: the first insert that relied on the default failed with unknown function: now(). The test suite created tables from models, so it never exercised this path. The audit caught it during the end-to-end run.
- The original FastAPI config test inherited the developer .env, so "missing key" could not be tested. That was a test-isolation bug, not a product feature.

2.3 KEEP / MODIFY / REMOVE / ADD

Path or area	Action	Why
backend/django/config/settings.py, config/urls.py	MODIFY	Register patients and reports; keep JWT, CORS, SQLite and telemetry settings
backend/django/accounts/models.py	MODIFY	Keep custom user and password hashing; reduce roles to operator/admin; delete profile models
backend/django/accounts/serializers.py, views.py, urls.py	MODIFY	Keep login/refresh/me; delete doctor and patient-account endpoints
backend/django/accounts/admin.py	MODIFY	Keep Django Admin for technical account administration
backend/django/accounts/permissions.py	REMOVE	Role classes existed only to separate doctors from patients
backend/django/accounts/migrations/0001..0003	KEEP	Historical migrations; never deleted or rewritten
backend/django/accounts/migrations/0004, 0005, 0006	ADD	Collapse roles, remove doctor/patient profile tables after FK swap, disable legacy patient logins
backend/django/patients/	ADD	Standalone permanent patient registry, search, duplicate warning, admin
backend/django/devices/models.py and serializers/views	MODIFY	Keep UUID, owner and token; point assigned_patient at the registry; remove role checks
backend/django/devices/migrations/0001, 0002	KEEP	Historical device schema and assignment field
backend/django/devices/migrations/0003	ADD	Repoint assigned_patient to patients.Patient with IDs preserved
backend/django/monitoring/models.py	MODIFY	Keep Session; add sample rate, notes, timestamps and SessionAnalysis
backend/django/monitoring/serializers.py, views.py, urls.py	MODIFY	Remove doctor scoping; add patient filter, stop/analyse/analysis/report actions
backend/django/monitoring/telemetry.py	MODIFY	Generic Django-to-FastAPI calls; no doctor_ids; analysis and ECG fetch helpers
backend/django/monitoring/analysis.py	ADD	Orchestrate FastAPI compute, store the exact snapshot, record failures
backend/django/monitoring/migrations/0001	KEEP	Historical session table
backend/django/monitoring/migrations/0002	ADD	Session -> registry FK, analysis table, new session fields
backend/django/reports/	ADD	Versioned Report model, PDF writer, generation service, authorized download
backend/django/scripts/mint*.py	MODIFY	Create registry patients, sync minted sessions to FastAPI, emit patient code
backend/django/conftest.py, tests	MODIFY/ADD	Operator/admin/patient/device fixtures; doctor workflow tests replaced
backend/fastapi/app/core/security.py	MODIFY	Accept operator/admin tokens; reject legacy patient/doctor tokens
backend/fastapi/app/models.py	MODIFY	Drop doctor_ids; add versioned AnalysisResult
backend/fastapi/app/registry.py	MODIFY	Any authenticated operator may view a session; device may view its own
backend/fastapi/app/routes/sensor.py	MODIFY	Device status is clinic-wide; no role branching
backend/fastapi/app/routes/internal.py	MODIFY/ADD	Registry without doctor IDs; internal full-ECG fetch for PDF
backend/fastapi/app/routes/analysis.py	ADD	Stored analysis, latest analysis and unpersisted live analysis
backend/fastapi/app/dsp.py	ADD	Pure-Python filtering, R-peaks, heart rate, RR metrics, quality
backend/fastapi/app/inference.py	ADD	Stable model interface; explicit unavailability when no model
backend/fastapi/app/analysis.py	ADD	Combine DSP, inference, findings and limitations
backend/fastapi/app/schemas.py	MODIFY	Remove doctor_ids; add analysis and internal ECG schemas
backend/fastapi/alembic/versions/0001	MODIFY	Portable CURRENT_TIMESTAMP defaults for fresh installs
backend/fastapi/alembic/versions/0002, 0003	ADD	Analysis table and drop doctor_ids; repair timestamp defaults
backend/fastapi/tests/	MODIFY/ADD	New access model, DSP, analysis, inference and migration coverage
frontend/src/App.tsx	MODIFY	Six focused routes; remove role gates and doctor/admin pages
frontend/src/api/types.ts, client.ts	MODIFY	Registry, sessions, analysis, reports, runtime telemetry base
frontend/src/components/AppShell.tsx	MODIFY	Dashboard/Patients/Settings navigation; AI model status
frontend/src/components/PatientForm.tsx	ADD	Register/edit with duplicate warning and confirm-anyway
frontend/src/components/StartSessionDialog.tsx	MODIFY	Patient + device recording start; no doctor language
frontend/src/components/SessionList.tsx	MODIFY	Recording, analysis and report state per session
frontend/src/components/VitalCard.tsx, Sparkline.tsx	REMOVE	Four-vital dashboard no longer part of the ECG workflow

Path or area	Action	Why
frontend/src/pages/DashboardPage.tsx, PatientsPage.tsx	MODIFY	Search/register/start workflow and recording history
frontend/src/pages/PatientProfilePage.tsx, MonitorPage.tsx, AnalysisPage.tsx, SettingsPage.tsx	ADD	Required screens A-F
frontend/src/pages/AdminPatientsPage.tsx, LivePage.tsx, RecordingPage.tsx, SessionsPage.tsx	REMOVE	Replaced by the focused workflow
frontend/src/pages/ReportsPage.tsx, DevicesPage.tsx	ADD	Dedicated report history and device-management pages
frontend/src/index.css, LoginPage.tsx, AppShell.tsx, monitor/analysis pages	MODIFY	PulseTrace ECG light workspace, navy sidebar, patient table, paper ECG grid, four-card analysis layout
frontend/src/auth/Guards.tsx	MODIFY	Authentication guard only; no RoleGate
frontend/src/pages/LoginPage.tsx	MODIFY	Operator/admin demo accounts and ECG wording
frontend/src/lib/format.ts, ecg.ts, index.css	MODIFY	Patient formatting, waveform helpers, workflow styles
simulator/	KEEP	Ingestion contract and signal generation unchanged
simulator/tests/	MODIFY	Remove doctor_ids from the registry fixture
docs/api-contract.md	MODIFY	New roles, removed doctor cache, analysis endpoints
docs/phase-4-notes.md	MODIFY	Marked historical; points to this document

2.4 Migration risks and mitigations

The risky part was changing foreign keys while keeping every session and device attached to the right person. The strategy was:

- Create patients.Patient first.
- Copy every PatientProfile into a Patient with the same primary key, a new permanent patient ID, the legacy user link and the original creation time.
- Change roles in a separate migration (doctor/patient -> operator, admin stays admin).
- Repoint Device.assigned_patient and Session.patient to patients.Patient while the old tables still exist.
- Only then delete DoctorAssignment, DoctorProfile and PatientProfile.

The local database was backed up before the migration. After migration, all five legacy patient profiles were present in the registry, six sessions still pointed at the same patient keys, and the one dedicated device still pointed at its patient. No sessions, devices or patients were deleted.

3. Target architecture

Service	Owns	Never owns
Django	Operators/admins, JWT, permanent patients, devices, sessions, analysis snapshots, report metadata and PDF bytes	ECG sample storage, DSP inference
FastAPI	Scalar telemetry, ECG chunks, session registry, DSP, optional inference, analysis versions	Patient identity, operator accounts, PDF metadata
React	Workflow, search, live chart, analysis presentation, report download	Any clinical computation or fabricated prediction
Simulator	ESP32-compatible ingestion traffic	Patient identity or diagnosis

The two services share identifiers, not a database. No FastAPI foreign key points at a Django table. Django mints a short-lived admin token for internal work; the browser uses operator tokens.

4. Django backend

4.1 Permanent patient registry

patients.Patient has an internal integer key and a public, permanent patient_id such as PT-E28319D9. It stores full name, phone, normalised phone, email, date of birth or age, sex, notes, timestamps and a nullable link to a legacy login account. The phone number is not a key and is not unique: the same number can legitimately belong to two people. A duplicate phone during registration returns 409 duplicate_patient with the matches; the operator can open the existing profile or explicitly confirm creation. Search covers name, phone and patient ID. Patients do not have login accounts.

4.2 Authentication and roles

Authentication is unchanged: SimpleJWT access/refresh tokens with user_id and role. Roles are now operator and admin. Doctor accounts that existed before the refactor keep working as operators; legacy *patient* logins are disabled rather than promoted, because promoting a patient account would expose the whole registry. Django

Admin remains for technical administration but is not the operator interface. Every API view requires authentication; there is no doctor/patient object-level scoping to maintain.

4.3 Devices

Device UUIDs, owner and one-time tokens are preserved. A device may be dedicated to one patient or shared (assigned_patient = NULL). Registration, token rotation, dedication and enable/disable are explicit operations. Heartbeat/last-seen remain FastAPI's truth.

4.4 Sessions and analysis persistence

Session retains its UUID, patient, device, operator, start/end time and status, and adds sample rate, notes and timestamps. The database still enforces one active session per device with a partial unique index. SessionAnalysis stores the FastAPI result exactly, plus status, model version, timestamp and error. A failed compute call is stored as failed; it is never replaced with a made-up result. Session actions: create, list/filter, stop, analyse, read analysis, generate/read report, list report versions.

4.5 Reports

reports.Report stores a report ID, session, patient, version, exact analysis snapshot, PDF bytes, size, SHA-256 checksum, author and timestamp. Generation is idempotent without regenerate=true; regeneration creates a new version. Downloads go through /api/reports/<id>/download/ with authentication, application/pdf, an attachment filename and no-store. PDFs are never exposed from a public media path.

The PDF writer is dependency-free (PDF 1.4, Helvetica/Courier, vector waveform). It contains patient and session identity, recording times, device, sampling frequency, quality, measurements, waveform, AI result or explicit unavailability, findings, limitations and the prominent screening-aid statement. Long recordings are decimated for drawing; the stored snapshot is bounded separately.

4.6 Migrations applied

Migration	Effect
patients.0001_initial	Create the registry
patients.0002_migrate_legacy_patients	Copy profiles with preserved IDs and permanent patient IDs
accounts.0004_remove_profiles_and_simplify_roles	Map doctor/patient -> operator; role choices
devices.0003_assigned_patient_registry	Device FK -> registry
monitoring.0002_patient_registry_analysis	Session FK -> registry; analysis table; new fields
accounts.0005_remove_doctor_and_patient_profiles	Drop doctor/patient profile tables after FK swap
accounts.0006_disable_legacy_patient_logins	Disable pre-refactor patient logins; doctor accounts remain operators
reports.0001_initial	Report table and version constraint
fastapi 0002_analysis	Analysis table; drop doctor_ids
fastapi 0003_timestamp_defaults	Repair SQLite/PostgreSQL timestamp defaults

Commands used: python manage.py migrate, python -m alembic upgrade head.

5. FastAPI, DSP and AI

5.1 Ingestion contract

POST /sensor/connect, POST /sensor/data, POST /sensor/ecg, the WebSocket /ws/sensor, chunk reads and the simulator payloads are unchanged except that user roles are operator/admin. Device tokens, payload limits, idempotency constraints, duplicate handling, revocation and heartbeat semantics are intact. known_sessions no longer carries doctor_ids.

5.2 Signal processing

The DSP is pure Python so it runs on the same small deployment without NumPy/SciPy:

- Validate sample rate, chunk consistency, duration, finite values, flatline and clipping.
- Apply a 0.5 Hz high-pass (baseline wander), 50/60 Hz notch and 40 Hz low-pass, forward-backward for zero phase.
- Detect R-peaks on a 5-15 Hz band-passed derivative-square envelope with a 250 ms refractory window; refine each peak to the largest deflection within ±80 ms.
- Compute heart rate from the median RR interval, RR intervals, mean, standard deviation, RMSSD, pNN50 and beat count.

- Score quality 0-100 from missing samples, flatline, clipping, chunk gaps, beat count and RR outlier ratio. Below the configured threshold, all measurements are withheld and the analysis is unusable with reasons.

5.3 Inference interface

ECG_MODEL_PATH optionally points to a validated JSON linear classifier (classes, feature_names, mean/scale/coefficients/intercept). The service produces softmax probabilities only when that model exists and the signal is usable. No model is bundled. With no model, ai.available=false, predicted_class=null, probabilities=null, and a reason is returned. The API, UI and PDF all state that AI is unavailable. No synthetic ECG run is presented as proof of diagnostic accuracy.

6. React frontend

Required routes exist: Dashboard (/), Patients (/patients), Patient Profile (/patients/:id), ECG Monitor (/monitor and /monitor/:id), Reports (/reports), Analysis & Report (/sessions/:id), Devices (/devices) and Settings (/settings). The visual layer now follows the supplied PulseTrace ECG reference: a light clinical workspace, dark navy sidebar, paper-grid ECG waveform, patient table with filters/pagination, and the four-card analysis layout. The dashboard combines patient search, registration, recording start, recent recordings, device/service state and analysis/AI status. The monitor shows the rolling waveform, socket state, recording controls, live quality and heart rate from the backend's unpersisted live endpoint, and a simulator command. The analysis page shows the stored waveform, measured parameters, AI result or unavailability, findings, limitations, report generation and download.

Removed: doctor lists, doctor profiles, doctor-specific dashboards, assignment interfaces, role-gated navigation, admin patient creation, the separate sessions administration screen, four-vital cards and their sparklines. The design system, uPlot chart, auth flow, modal, badges and banners were reused.

7. End-to-end verification

Environment: Django on 8000, FastAPI on 8001, SQLite databases, simulator synthetic ECG at 200 Hz.

- Operator login succeeded; /api/auth/me/ returned role operator.
- seed_demo created operator/admin, three patients and a simulator device.
- A fresh session was minted and registered with FastAPI.
- Simulator ran 10 s with --ecg: 12 chunks, 2,400 samples, 36 scalar readings, 0 duplicates, 0 failures, 51 HTTP 200 responses.
- Session stop returned completed; analysis returned completed, 15 beats, 72.9 bpm, 823.6 ms mean RR, quality 100/100.
- AI result was explicitly unavailable (no model configured), with no class or probability.
- Report generation stored version 1, then version 2 after regeneration; download returned application/pdf.
- pdfinfo reported PDF 1.4, A4, 2 pages; pdftotext found the patient/session details, measurements, waveform label, AI-unavailable statement, limitations and the screening-aid banner.
- Patient history returned both sessions and both report versions under the same permanent patient ID.
- Duplicate phone registration returned HTTP 409 with the existing patient.
- Unauthenticated patient/session/device/report requests return 401 (tested).

Test suites after the refactor:

Suite	Result
Django	39 passed
FastAPI	105 passed
Simulator	96 passed
Frontend	tsc -b --noEmit, ESLint and vite build pass

The original FastAPI suite had one pre-existing failure caused by .env leaking into a subprocess; that test is now hermetic and passes.

8. API changes

Removed: GET /api/doctors/, PUT /api/patients/<id>/doctors/, doctor assignment fields, patient-account creation semantics and role-gated session/device rules. Patient creation is now registry creation with an optional confirm_duplicate.

Changed: user token roles {operator, admin}; legacy patient/doctor tokens are rejected and require a new login; known_sessions.doctor_ids removed; all authenticated operators share the clinic's patient/device/session scope.

Added: GET/POST /api/sessions/<id>/analysis, GET /api/sessions/<id>/reports, POST/GET /api/sessions/<id>/report, GET /api/reports/<id>/download, GET /api/patients/duplicates/, GET /api/patients/<id>/sessions, FastAPI POST/GET /sessions/<id>/analysis, GET /sessions/<id>/analysis/live, GET /internal/sessions/<id>/ecg.

Unchanged: every simulator and ESP32 ingestion endpoint and payload, device tokens, the WebSocket frame shape for readings and ECG chunk announcements, duplicate/idempotency behavior.

9. Manual test steps

```
cd backend/django
python manage.py migrate
python manage.py seed_demo
python manage.py runserver 127.0.0.1:8000

cd backend/fastapi
python -m alembic upgrade head
python -m uvicorn app.main:app --host 127.0.0.1 --port 8001

cd frontend
npm install
npm run dev
```

Sign in as operator / demo12345, open Patients, register or select a patient, start a recording, run the simulator with the device token and session id, watch the monitor, stop, review analysis and download the PDF. Repeat with the same patient to confirm history reuse. Use a flat or short signal to confirm measurements are withheld with reasons.

10. Remaining limitations

- No validated AI model is bundled; AI is unavailable by design until one is configured and independently validated.
- All authenticated operators can see all patients. A future multi-clinic deployment needs an explicit organisation/tenant scope.
- Analysis is synchronous in the request; a long recording would benefit from a queue/worker.
- WebSocket fan-out is in-process; multiple FastAPI replicas need Redis pub/sub.
- Chunk and reading listings are unpaginated.
- The PDF is vector and readable but intentionally simple; it is not a regulatory-format clinical report.
- A token is stored in local storage and the WebSocket query string, as before; one-time socket tickets and httpOnly cookies remain hardening work.
- Some obsolete CSS selectors from the removed admin screens remain in index.css; they are not referenced by any route or component.

11. Conclusion

The application is now an ECG screening workflow rather than a hospital dashboard. The patient record is permanent and independent of login, historical recordings and reports are reachable on later visits, the simulator still works with the original contract, DSP output is measured rather than invented, AI absence is explicit, and the PDF is generated from the stored result. The remaining work is model validation and operational hardening, not a missing workflow.